

Li Haoran

138-0000-0000 | lihaoran@163.com

Job Intent: LLM Application Development / Agent Development Intern

Education

Institute of Computing Technology, Chinese Academy of Sciences	Artificial Intelligence · Master's	2026.9–2029.6
Hohai University (211)	Computer Science and Technology · Bachelor's	2022.6–2026.9
GPA: 4.8/5.0 (Top 5%) CET-6: 523 National Scholarship , Excellent Student Scholarship, etc. First Prize, National College Student Mathematics Competition National Third Prize, China College Student Service Outsourcing Innovation and Entrepreneurship Competition MCM/ICM H Prize		

Project Experience

Salvo — E-commerce After-Sales Intelligent Customer Service Agent Platform	2026.5–2026.8
GitHub: github.com/li02045617/Salvo	

Project Summary: For e-commerce after-sales scenarios, designed and implemented an intelligent customer service platform based on a **LangGraph**-controlled single-Agent architecture, connecting the complete loop of intent recognition, layered memory, RAG tool retrieval, human-handoff tickets, monitoring feedback and automatic evaluation, solving problems such as inaccurate recognition of complex intents, weak knowledge recall, easily lost multi-turn context, unobservable runtime state, and difficulty evaluating and optimizing after launch.

Tech Stack: Python · FastAPI · LangGraph · Next.js/React · Redis · ChromaDB · RAG (BM25+RRF)

Project Content:

- 1. Agent Runtime & Business Flow:** Based on **LangGraph**, designed a controlled single-Agent workflow that organizes intent recognition, tool calling, result verification and failure downgrade / human handoff into a controllable execution chain; the tool layer encapsulates capabilities uniformly through the **MCP** specification, providing **caching, circuit breaking, timeout and downgrade** capabilities; through tool whitelists and conditional constraints on Agent behavior, supports multi-task execution and exception fallback; the front end implements an **Agent-trace** trajectory panel to track Agent decisions and tool execution in real time.
- 2. Multi-Route Fusion Intent Recognition:** Combined LLM semantic classification, Embedding similarity and keyword rules for intent voting, covering **10 types** of after-sales intents such as orders, refunds, logistics and returns/exchanges and **4 confidence tiers**, and used an **LRU cache** to reduce high-frequency repeated inference calls; intent results determine the task type, and the task type dynamically controls the tool whitelist, narrowing the Agent's action space and avoiding irrelevant or unauthorized tool calls.
- 3. RAG Hybrid Retrieval Pipeline:** Designed and implemented **Hybrid Retrieval**, fusing **BM25** keyword recall and **ChromaDB** vector recall through **RRF**, achieving Query Rewrite, parallel recall, deduplication, LLM reranking and metadata filtering, together with a **TTL cache and circuit-breaking/downgrade mechanism**; built a business knowledge base covering **9** after-sales knowledge domains (**37** documents), registered as an MCP tool for the Agent to retrieve on demand.
- 4. Retrieval Evaluation & Optimization:** Built a retrieval evaluation set of **35** manually annotated Queries, comparing three strategies—direct / hybrid / rewrite—on Recall, Precision, MRR and NDCG, and raising **Recall@5 from 0.31 to 0.74** through experimental iteration; supplemented end-to-end trial validation of the Agent's knowledge-retrieval tool calls and recall results, and combined it with LLM-as-Judge for end-to-end evaluation of dialogue quality on 20 customer-service cases.
- 5. Layered Memory Management:** Based on Redis and vector storage, designed a three-layer memory mechanism of short-term working memory, long-term episodic memory and user profile; through context compression and dynamic splicing, controls Token spend, and supports cross-turn retrieval of historical information and reuse of the user profile.

Mini-dsh — A Coding Agent Harness Based on a Plugin Architecture	2026.8–Present
GitHub: github.com/li02045617/mini-dsh	

Project Summary: Drawing on the "everything is a plugin" design philosophy of DeepSeek Harness, implemented a lightweight Coding Agent Harness from scratch in TypeScript/Node.js, modularizing the Agent's core capabilities into replaceable plugins, and quantifying the impact of different Harness mechanisms on Agent execution results through Benchmarks and ablation experiments.

Tech Stack: TypeScript · Node.js · DeepSeek API · JSONL

Project Content:

- 1. Plugin-based Agent Runtime:** Implemented MiniContext, PluginRegistry and ServiceRegistry around the **MiniPlugin/Service protocol**, using **Plugin/Service** abstractions to decouple Model, Tool, Session, Permission, Event, Context Manager and Agent Loop; through dependency validation, Service injection and lifecycle management, plugins run and can be independently replaced without modifying the execution loop.
- 2. Agent Loop & Tool Execution:** Implemented multi-state progression of Model → Tool → Result, configured with **64 model-request rounds, 128 tool calls, a 10-minute task timeout** and AbortController cancellation; based on the native **Tool-Calling** capability, abstracted a unified ModelRequest/ModelResponse, mapped the structured interface of model returns to a unified tool execution interface, and provided file operations and run_command basic tools.
- 3. Session & Context Management:** Persists complete messages, tool calls and results with append-only **JSONL**, supporting Session recovery and /continue after model budget is exhausted; the Context Manager triggers model summarization at an **80% threshold**, retaining only the latest 12 messages and truncating large tool results to mitigate context bloat in long-range tasks; additionally designed a **Prompt Optimizer** plugin that converts vague requirements into executable Worker Prompts, reducing the impact of unclear task constraints on execution.
- 4. Benchmark Design & Ablation Experiments:** Built a self-designed **20-task Benchmark + automatic Evaluator** covering code comprehension, bug fixing, feature implementation, refactoring and test writing; with unified model, permissions and budget, ran comparative experiments across **Baseline / Context Manager / Prompt Optimizer / Full Harness**, computing 6 categories of metrics including **success rate, Model Rounds, Tool Calls/Errors, Token and Latency**: Full Harness reached a **90% (18/20)** success rate vs. **75% (15/20)** for the Baseline, while the average number of model responses, the number of tool failures and Token consumption all decreased, validating the effectiveness of the Harness.

Personal Skills

Familiar with **Python / TypeScript** and Agent, RAG, Tool Calling and LLM application development;
Familiar with technology stacks such as **FastAPI, LangGraph, Redis and Docker**;
Proficient in using AI coding tools such as **Claude Code, Codex and dsh** to assist development, debugging and engineering iteration.